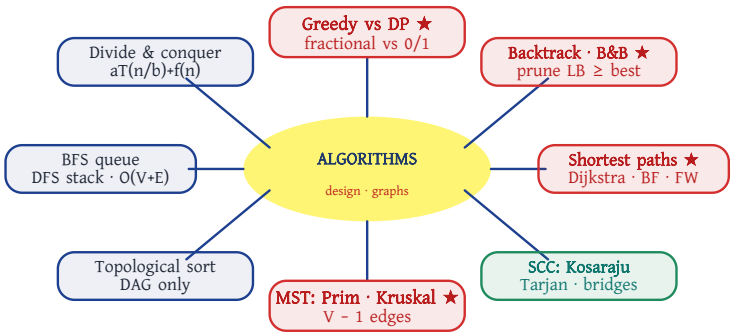
Algorithm Design Methods & Graph Algorithms

Divide & conquer · greedy · DP · backtracking · branch & bound · BFS/DFS · topological sort · SCC · MST · shortest paths: exam-ready in eleven sheets



↑ at a glance: red = high-yield, green = DFS family

Syllabus coverage

The Data Structures and Algorithms line reads: "...**Methods for the design of efficient algorithms: divide and conquer, greedy method, dynamic programming, back tracking, branch and bound; Basic search and traversal techniques; Topological sorting; Connected components, spanning trees, shortest paths;** Flow algorithms; ..." This set takes everything from the design methods to shortest paths. The **master theorem** and sorting details sit in the recursion & sorting lesson, the **heap** behind Dijkstra/Prim in the heaps lesson, **flow** in the flow & approximation lesson and **NP-completeness** in the last lesson; graph-theory basics are chapter 13.

High-yield list

① Merge sort $2T(n/2)+n = \Theta(n \log n)$
② Strassen $O(n^{\log_2 7}) \approx O(n^{2.81})$
③ Greedy by v/w: **fractional** knapsack only
④ DP = overlapping + optimal substructure
⑤ Floyd-Warshall & Bellman-Ford are DP
⑥ Backtracking = DFS of state space
⑦ B&B (min): prune when LB ≥ best
⑧ BFS/DFS: $O(V+E)$ list, $O(V^2)$ matrix
⑨ Topological order ⇔ DAG
⑩ MST has exactly V − 1 edges
⑪ Dijkstra: no negative edges
⑫ Bellman-Ford $O(VE)$, finds neg. cycle

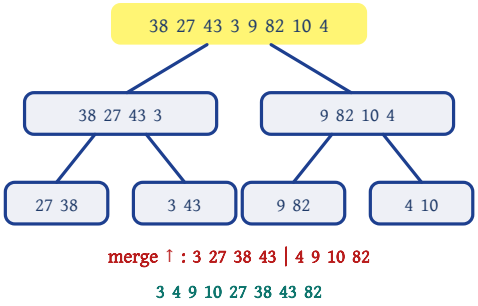
Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

Contents

✓ Divide & conquer ... p. 2
✓ Greedy method ... p. 3
✓ DP: Fibonacci, LCS ... p. 4
✓ DP: knapsack, matrix chain ... p. 5
✓ Backtracking, branch & bound ... p. 6
✓ BFS, DFS, topological sort ... p. 7
✓ Components, SCC, MST ... p. 8
✓ Shortest paths ... p. 9
✓ Night-before sheet ... p. 10
✓ Practice MCQs + written ... p. 11

1 Divide and Conquer: Split, Solve, Combine

Definition: **Divide and conquer** solves a problem by (1) **Divide**: splitting it into a smaller instances of the **same** problem, (2) **Conquer**: solving each one recursively (tiny instances directly), (3) **Combine**: merging the sub-answers. The sub-problems are **independent** (they do not overlap), which is what separates it from dynamic programming. Its cost is the recurrence **T(n) = aT(n/b) + f(n)**: a sub-problems of size n/b, f(n) = divide + combine work.



merge sort: $\log_2 n$ levels × n work per level = n log n

Algorithm	Recurrence	Result
Binary search	T(n/2) + 1	Θ(log n)
Merge sort	2T(n/2) + n	**Θ(n log n)**
Quick sort best/avg	2T(n/2) + n	Θ(n log n)
Quick sort worst	T(n−1) + n	$\Theta(n^2)$
Max–min (n items)	2T(n/2) + 2	3n/2 − 2 comps
Karatsuba	3T(n/2) + n	$n^{1.585}$
Strassen	7T(n/2) + n^2	$\mathbf{n^{2.81}}$
Closest pair	2T(n/2) + n	Θ(n log n)

Worked example ✍ Strassen: Ordinary block multiplication of n×n matrices splits each into four n/2 blocks and needs **8** block products: T(n) = 8T(n/2) + n^2 = $\Theta(n^3)$. Strassen rearranges the work into **7** products (and 18 additions): T(n) = 7T(n/2) + $\Theta(n^2)$. Since $\log_2 7 \approx 2.807 > 2$, the leaves dominate: $\mathbf{\Theta(n^{\log_2 7}) \approx O(n^{2.81})}$.

Solving it fast: Compare f(n) with $n^{\log_b a}$. Leaves heavier → $\Theta(n^{\log_b a})$; equal → $\Theta(n^{\log_b a} \cdot \log n)$; root heavier → Θ(f(n)). Full master theorem with its regularity condition: recursion & sorting lesson.

Exam trap: (i) Quick sort is D&C even though its **combine** step is empty — the work is in the **partition**. Merge sort is the opposite: trivial divide, real combine. (ii) Binary search keeps **one** half (a = 1), so it is sometimes called "decrease and conquer".

Memory trick: "**D-C-C: Divide, Conquer, Combine.**" Strassen: "**seven, not eight**" → $\log_2 7 \approx 2.81$. Karatsuba: "**three, not four**" → $\log_2 3 \approx 1.585$.

When D&C fits: sub-problems are of the same kind, independent and roughly equal in size. If the same sub-problem keeps reappearing (naive Fibonacci), switch to **DP**.

Worked example ✍ binary search: A = [3, 9, 14, 21, 27, 35, 42], key 27. mid = A[3] = 21 < 27 → right half [27, 35, 42]; mid = 35 > 27 → left half [27]; found. **3 comparisons** = $\lfloor \log_2 7 \rfloor + 1$, the worst case for n = 7. Needs a **sorted** array with random access.

```
MergeSort(A, l, r):
  if l < r:
    m = (l + r) / 2
    MergeSort(A, l, m)       // conquer
    MergeSort(A, m + 1, r)   // conquer
    Merge(A, l, m, r)        // combine, O(n
```

2 The Greedy Method: Best Choice Now, Never Undo It

Definition: A **greedy algorithm** builds a solution one piece at a time, always taking the choice that looks best **right now** (the local optimum) and never reconsidering it. It gives the global optimum only when the problem has both the **greedy-choice property** (some optimal solution starts with the greedy choice) and **optimal substructure** (after that choice, the rest is an optimal solution of the smaller problem). Otherwise it gives a fast but possibly wrong answer.

Worked example ✍ fractional knapsack, W = 50: Items (value, weight): I_1 (60, 10), I_2 (100, 20), I_3 (120, 30). Ratios v/w = **6, 5, 4**. Sort by ratio and fill: take all of I_1 (w 10, v 60), all of I_2 (w 30, v 160), then 20/30 of I_3 → 120 × 2/3 = 80. **Total = 240**, optimal. Cost: O(n log n) for the sort.



same items, two problems, two answers

Exam trap: greedy fails here: (i) **0/1 knapsack** (items cannot be split): by ratio, greedy takes $I_1 + I_2 = 160$, but the optimum is $I_2 + I_3 = 220$. Use DP. (ii) **Coin change** with coins {1, 3, 4}, amount 6: greedy gives 4 + 1 + 1 (3 coins), optimum is 3 + 3 (2 coins). Greedy is optimal only for "canonical" systems like 1, 2, 5, 10.

Greedy algorithm	Greedy rule	Time
Fractional knapsack	highest value/weight first	O(n log n)
Activity selection	earliest **finish** time first, skip clashes	O(n log n)
Huffman coding	merge the two least-frequent nodes (min-heap)	O(n log n)
Job sequencing with deadlines	highest profit first, latest free slot ≤ deadline	$O(n^2)$
Prim / Kruskal (MST)	cheapest safe edge	p. 8
Dijkstra	settle the closest unsettled vertex	p. 9

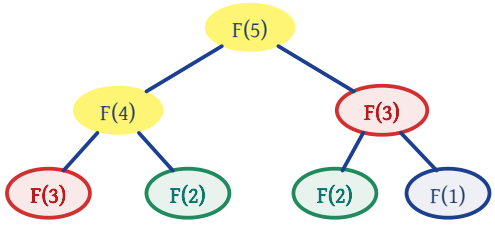
Worked example ✍ activity selection: (start, finish): A(1,4) B(3,5) C(0,6) D(5,7) E(3,9) F(5,9) G(6,10) H(8,11) I(8,12) J(2,14) K(12,16). Sorted by finish: pick A (ends 4) → first start ≥ 4 is D (ends 7) → H (ends 11) → K. **4 activities** {A, D, H, K}. Sorting by *start* or by *shortest* duration does not always work.

	Greedy	DP
Choices	one, never undone	all, best kept
Needs	greedy-choice + opt. substr.	overlap + opt. substr.
Speed	faster	slower, table
Knapsack	fractional ✓	0/1 ✓

Memory trick: "**Greedy grabs, DP deliberates.**" Divisible goods (gold dust) → greedy; indivisible goods (gold bars) → DP.

3 Dynamic Programming: Solve Each Sub-problem Once

Definition: **Dynamic programming** (DP) breaks a problem into sub-problems that **overlap**, solves each one only once and stores the answer in a table. It needs two properties: **overlapping sub-problems** (the same sub-problem recurs many times) and **optimal substructure** (an optimal solution is built from optimal sub-solutions). Steps: define the sub-problem → write the recurrence → fix the base cases → fill the table in a safe order → trace back the answer.



F(3) twice, F(2) three times ...
naive calls grow like $\varphi^n \approx 1.618^n$

same colour = same sub-problem solved again

	Memoization	Tabulation
Direction	**top-down**: recursion + cache	**bottom-up**: loops
Solves	only sub-problems needed	every table cell
Overhead	call stack	none; space can shrink

Fibonacci table: F[i] = F[i−1] + F[i−2], F[0] = 0, F[1] = 1.

i	0	1	2	3	4	5	6
F	0	1	1	2	3	5	8

Naive recursion: exponential; DP: **O(n) time**, O(1) space with two variables.

Longest common subsequence (LCS)

Recurrence: c[i][j] = 0 if i = 0 or j = 0; **c[i−1][j−1] + 1** if $x_i = y_j$; else **max(c[i−1][j], c[i][j−1])**. Time and space **O(mn)**. A subsequence keeps order but need not be contiguous (a substring must be).

Memory trick: "**Match → diagonal + 1; mismatch → max of up and left.**" Trace back from the bottom-right: diagonal moves on matches spell the LCS.

Worked example ✍ **X = ABCB, Y = BDCAB:**

	Ø	B	D	C	A	B
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	**3**

LCS length **3** = "BCB" (B...C...B in both).

Exam trap: (i) DP is not "recursion with a table" for every problem: without **overlapping** sub-problems (merge sort), a table saves nothing — that is plain D&C. (ii) LCS of lengths m, n costs O(mn), not O(m + n). (iii) Memoization and tabulation have the **same** asymptotic time; they differ in order and overhead.

4 DP Classics: Knapsack and Matrix Chain

0/1 knapsack recurrence: K[i][w] = best value using items 1..i with capacity w.
K[i][w] = K[i−1][w] if w_i > w; else **max(K[i−1][w], v_i + K[i−1][w − w_i])**.
Time **O(nW)** — **pseudo-polynomial** (polynomial in the *value* W, exponential in its bit length).

Exam trap: the 1-D version: With one array, K[w] = max(K[w], v_i + K[w − w_i]) is correct only if w runs **downward from W to w_i**. Running w upward reuses item i in the same pass, which solves the **unbounded** knapsack (unlimited copies) instead.

Worked example ✍ W = 5: (w, v): 1 (2, 3) · 2 (3, 4) · 3 (4, 5) · 4 (5, 6)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	**7**

K[2][5] = max(3, 4 + K[1][2]) = 7. Answer **7** = items 1 + 2 (weight 5).

Matrix-chain multiplication (MCM)

Problem: Matrix A_i is $p_{i-1} \times p_i$. Choose the parenthesisation of $A_1A_2...A_n$ with the fewest scalar multiplications (a p×q times q×r product costs **pqr**). The order changes the cost, never the result. Recurrence: **m[i][j] = min over i ≤ k < j of m[i][k] + m[k+1][j] + $p_{i-1}p_kp_j$**, m[i][i] = 0. Time **O(n^3)**, space O(n^2). The number of parenthesisations is the Catalan number C_{n-1}, so brute force is exponential.

Worked example ✍ 3 matrices: A_1 10×30, A_2 30×5, A_3 5×60.
$(A_1A_2)A_3$ = 10·30·5 + 10·5·60 = 1500 + 3000 = **4500**
$A_1(A_2A_3)$ = 30·5·60 + 10·30·60 = 9000 + 18000 = 27000
Same product, **6× the work** with the wrong order.

Worked example ✍ p = (5, 4, 6, 2, 7):

m	1	2	3	4
1	0	120	88	**158**
2		0	48	104
3			0	84

Best: **$((A_1(A_2A_3))A_4)$** = 48 + 40 + 70 = 158.

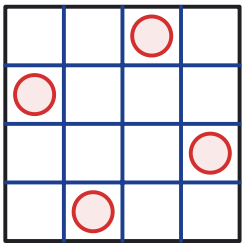
DP problem	Sub-problem	Time
Floyd-Warshall (all-pairs)	d_k[i][j]: paths using only vertices 1..k inside	O(V^3)
Bellman-Ford (single source)	shortest path using ≤ k edges	O(VE)
Edit distance	prefixes of both strings	O(mn)
Longest increasing subsequence	LIS ending at i	O(n^2) / O(n log n)
TSP (Held-Karp)	(set visited, last city)	O($n^2 \cdot 2^n$)

Memory trick: "**Floyd floats through every k**" — the outer loop is the intermediate vertex k, so Floyd-Warshall is DP, not greedy.

IT link: **diff**, Git and DNA alignment run LCS / edit distance; SQL optimisers choose a join order the way MCM chooses brackets.

5 Backtracking: DFS of the State Space

Definition: **Backtracking** builds a solution one component at a time along a **state-space tree**, exploring it **depth-first**. At each node a **bounding (feasibility) function** checks the partial solution; if it can no longer lead to a valid answer, the whole subtree is pruned and the algorithm steps back to the previous choice. Uses: N-Queens, sum of subsets, graph m-colouring, Hamiltonian cycle, Sudoku. Worst case stays exponential.



row 1 → col 3
row 2 → col 1
row 3 → col 4
row 4 → col 2

(3, 1, 4, 2) — mirror (2, 4, 1, 3)

4-Queens has exactly 2 solutions

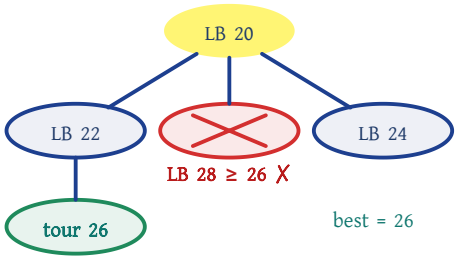
Worked example ✍ 4-Queens: One queen per row. Queens (r_1, c_1), (r_2, c_2) clash if $c_1 = c_2$ or $|r_1 - r_2| = |c_1 - c_2|$ (diagonal). Try row 1 col 1 → row 2 col 3 → row 3 has no safe column → back to row 2 col 4 → row 3 col 2 → row 4 dead → back up to row 1 col 2 → (2, 4, 1, 3) ✓. 8-Queens: **92 solutions** (12 distinct).

Sum of subsets: S = {3, 5, 6, 7}, target 15: include/exclude each item, prune when the sum passes 15 or cannot reach it. Only solution **{3, 5, 7}**.

6 Branch and Bound: Bounds Prune the Tree

Definition: **Branch and bound** (B&B) also searches a state-space tree, but for **optimisation**. Each live node gets a **bound** on the best answer reachable through it; if the bound cannot beat the **best complete solution so far**, the node is killed. Order: **FIFO** (BFS-like), **LIFO** or **LC (least-cost)** search with a priority queue. Uses: TSP (reduced cost matrix), 0/1 knapsack (fractional bound), job assignment.

Exam trap: the prune rule: For **minimisation** (TSP, assignment): LB(node) = least cost any completion can have; UB / best = cost of the best full solution found. Prune when **LB ≥ best** — a node with LB equal to best can only tie, so it is pruned too ("LB > UB" misses ties). For **maximisation** (0/1 knapsack): prune when the node's upper bound ≤ best value found.



minimisation: killed once LB ≥ 26

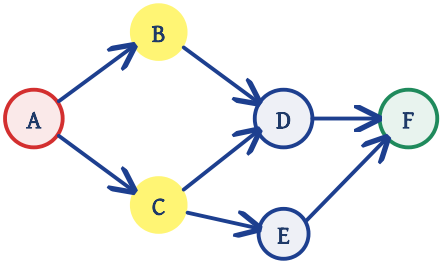
Memory trick: **"Backtracking asks 'is it legal?', B&B asks 'can it still win?'"** Backtracking = DFS + feasibility; B&B = any order (usually BFS / LC) + bound.

Method	Idea	Sub-problems	Typical examples
Divide & conquer	split, recurse, combine	independent	merge sort, binary search, Strassen
Greedy	local best, never undo	one chosen	fractional knapsack, Huffman, MST
Dynamic prog.	store sub-answers	**overlapping**	LCS, 0/1 knapsack, MCM, Floyd
Backtracking	DFS + feasibility	state-space tree	N-Queens, subset sum, colouring
Branch & bound	BFS / LC + bound	state-space tree	TSP, 0/1 knapsack, assignment

7 Graph Traversal: BFS and DFS

BFS: **Breadth-first search** uses a **queue**: visit the source, then all vertices at distance 1, then 2, ... It finds **shortest paths in unweighted graphs** (fewest edges), tests bipartiteness, and drives Edmonds-Karp and web crawlers.

DFS: **Depth-first search** uses a **stack** (or recursion): go as deep as possible, then back up. It records discovery/finish times and powers cycle detection, topological sort, SCC, articulation points and maze solving.



A→B, A→C, B→D, C→D, C→E, D→F, E→F

	BFS	DFS
Structure	queue (FIFO)	stack (LIFO) / recursion
Order from A (alphabetical)	**A B C D E F**	**A B D F C E**
Time	**O(V + E)** with adjacency list · $O(V^2)$ with adjacency matrix	
On a tree it gives	level-order traversal	pre-, in-, post-order traversals

DFS edge classes (directed): **Tree**: to a new vertex · **Back**: to an ancestor still open · **Forward**: to a finished descendant · **Cross**: to a vertex in another finished branch. Undirected DFS has only tree and back edges. **A back edge exists ⇔ the graph has a cycle.**

Exam trap: "DFS = pre/in/post-order, BFS = level order" is true for **trees** only; a general graph has no "in-order". BFS finds fewest-**edge** paths, not least-**weight** paths. A disconnected graph needs a fresh BFS/DFS from every unvisited vertex.

8 Topological Sorting

Definition: A **topological order** of a directed graph lists the vertices so that for every edge u → v, u comes before v. One exists **if and only if the graph is a DAG** (directed acyclic graph), and it is usually **not unique**. Uses: task scheduling, course prerequisites, build systems (make), spreadsheet recalculation, package installers.

Kahn's algorithm (BFS) ✍️: In-degrees A0 B1 C1 D2 E1 F2. Queue the 0s: A. Pop A → B, C drop to 0. Pop B → D = 1. Pop C → D = 0, E = 0. Pop D → F = 1. Pop E → F = 0. Pop F. Order **A B C D E F**. If fewer than V vertices come out, the graph has a cycle.

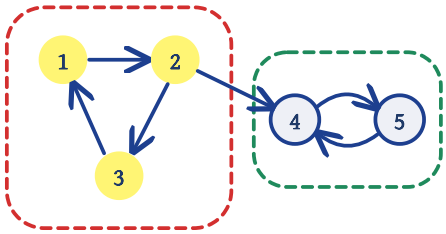
DFS method ✍️: Run DFS; when a vertex **finishes**, push it on a list. Finish order F, D, B, E, C, A → reversed: **A C E B D F**. Different from Kahn's, and equally valid. Both run in **O(V + E)**.

Memory trick: "**Kahn counts in-degrees, DFS reverses finish times.**" Prerequisite example: Math → Data Structures → Algorithms, Physics → Robotics gives Math, Physics, Data Structures, Algorithms, Robotics.

9 Connected and Strongly Connected Components

Undirected: A **connected component** is a maximal connected subgraph. Start a BFS/DFS from every unvisited vertex: number of starts = number of components, O(V + E).

Directed: A **strongly connected component** (SCC) is a maximal set where every vertex reaches every other. **Kosaraju**: (1) DFS on G, note finish order; (2) build the transpose G^T (reverse all edges); (3) DFS on G^T in decreasing finish time — each tree is one SCC. **Tarjan**: one DFS with a stack and low-link values; v is an SCC root when low[v] = disc[v]. Both **O(V + E)**.

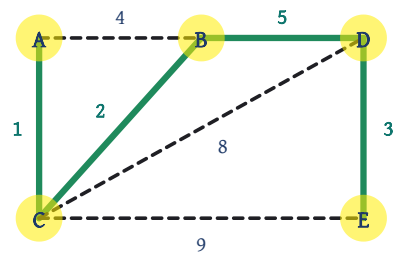


SCCs {1,2,3} and {4,5}

Exam trap: Shrinking every SCC to one node gives the **condensation**, which is always a DAG. An **articulation point** (cut vertex) is a vertex whose removal disconnects the graph; a **bridge** is such an edge. With DFS low values: the root is a cut vertex iff it has ≥ 2 DFS children; another vertex u is one iff some child v has low[v] ≥ disc[u]; edge (u, v) is a bridge iff low[v] > disc[u].

10 Minimum Spanning Tree: Prim and Kruskal

Definition: A **spanning tree** of a connected undirected graph contains all V vertices and exactly **V − 1 edges**, no cycle. A **minimum spanning tree** (MST) has the least total weight. **Cut property**: the lightest edge crossing any cut belongs to some MST — the reason both greedy algorithms are correct. Distinct weights ⇒ the MST is unique.



green = MST, weight 11

Worked example ✍️ same graph, two algorithms: Edges AB 4, AC 1, BC 2, BD 5, CD 8, DE 3, CE 9.

Kruskal (sort, add if no cycle): AC 1 ✓, BC 2 ✓, DE 3 ✓, AB 4 ✗ (cycle A-B-C), BD 5 ✓ → 4 edges, stop.

Prim from A (grow one tree): AC 1 → CB 2 → BD 5 → DE 3.

Both: **weight 1 + 2 + 3 + 5 = 11**, 4 = V − 1 edges.

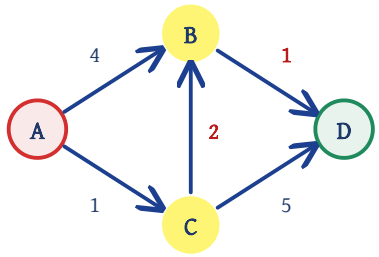
	Prim	Kruskal
How	grows one tree; priority queue of fringe vertices	sorted edges + **union-find** forest
Time	**$O(V^2)$** matrix · $O(E \log V)$ binary heap · $O(E + V \log V)$ Fib. heap	$O(E \log E)$ = **$O(E \log V)$**
Best for	**dense** graphs ($O(V^2)$ version)	**sparse** graphs

Exam trap: Dense graph ($E \approx V^2$): heap Prim costs $O(V^2 \log V)$, so the plain **$O(V^2)$ matrix Prim** wins. $\log E = O(\log V)$ since $E \le V^2$. MST ≠ shortest-path tree.

Memory trick: "**Prim = Proximity (grow from the tree), Kruskal = Kheapest edge anywhere.**"

11 Shortest Paths: Relax Until Stable

Relaxation: Keep d[s] = 0, others ∞, and **relax** edges: if d[u] + w(u, v) < d[v] then d[v] ← d[u] + w(u, v), parent[v] ← u. The algorithms differ only in **which** edges they relax and **in what order**.



directed, weights as drawn

Worked example ✍️ Dijkstra from A:

Settle	A	B	C	D
start	0	∞	∞	∞
A (0)	0	4	1	∞
C (1)	0	3	1	6
B (3)	0	3	1	4
D (4)	0	**3**	**1**	**4**

Final: **A 0, C 1, B 3, D 4**; path to D = A → C → B → D.

Dijkstra (greedy): Repeatedly **settle** the unsettled vertex with the smallest d and relax its out-edges; a settled distance is final. Needs **non-negative weights**. Time: $O(V^2)$ array · $O((V+E) \log V)$ binary heap · $O(E + V \log V)$ Fibonacci heap.

Exam trap: negative edge: A→B 2, A→C 3, C→B -2. Dijkstra settles B at 2 and never revisits it, but A→C→B = **1**. One negative edge is enough to break it; use Bellman-Ford.

Bellman-Ford (DP): Relax **every** edge V − 1 times (a shortest path has ≤ V − 1 edges). One more pass: if any edge still relaxes, a **negative-weight cycle** is reachable from the source. Handles negative edges. Time **$O(VE)$**.

Floyd-Warshall (DP): All pairs: for k = 1..V, for i, for j: d[i][j] = min(d[i][j], d[i][k] + d[k][j]). Time **$O(V^3)$**, space $O(V^2)$. Negative edges are fine; a negative cycle shows as d[i][i] < 0, and then shortest paths are undefined.

Algorithm	Scope	Method	Time	Negative edges?
BFS	1 source, unweighted	queue	$O(V + E)$	n/a
DAG shortest path	1 source, DAG	topo order	$O(V + E)$	yes
Dijkstra	1 source	greedy	$O((V+E) \log V)$	**no**
Bellman-Ford	1 source	DP	$O(VE)$	yes; **detects** neg. cycle
Floyd-Warshall	all pairs	DP	$O(V^3)$	yes; **no** neg. cycle

Exam trap: Bellman-Ford detects only **negative-weight** cycles (general cycle detection is DFS's job). Floyd-Warshall can reveal a negative cycle but not solve around it. Warshall (same loops, AND/OR) gives the **transitive closure**.

Memory trick: "**BFS for no weights, Dijkstra for positive, Bellman for negative, Floyd for all pairs.**" Complexities rise in the same order: $V+E \to E \log V \to VE \to V^3$.

Night-Before-the-Exam Sheet

Formulas & complexities:

D&C: $T(n) = aT(n/b) + f(n)$

Merge sort $2T(n/2)+n \to \Theta(n \log n)$

Binary search $T(n/2)+1 \to \Theta(\log n)$

Quick sort worst $\Theta(n^2)$

Strassen $7T(n/2)+n^2 \to n^{2.81}$

Karatsuba $3T(n/2)+n \to n^{1.585}$

LCS $O(mn)$ · edit distance $O(mn)$

0/1 knapsack $O(nW)$ pseudo-poly

MCM $O(n^3)$, cost pqr per product

Fibonacci DP $O(n)$, naive exponential

BFS/DFS $O(V+E)$ list, $O(V^2)$ matrix

Topological sort $O(V+E)$, DAG only

SCC (Kosaraju/Tarjan) $O(V+E)$

MST: $V - 1$ edges

Prim $O(V^2)$ / $O(E \log V)$ / $O(E+V \log V)$

Kruskal $O(E \log E) = O(E \log V)$

Dijkstra $O((V+E) \log V)$, $w \geq 0$

Bellman-Ford $O(VE)$

Floyd-Warshall $O(V^3)$, DP

DAG shortest path $O(V+E)$

Top 14 one-liners for MCQs

1. D&C sub-problems independent; DP ones overlap
2. Greedy needs greedy-choice + optimal substructure
3. Fractional knapsack → greedy by v/w
4. 0/1 knapsack → DP (greedy fails)
5. Huffman, Prim, Kruskal, Dijkstra = greedy
6. Floyd-Warshall, Bellman-Ford, LCS, MCM = DP
7. Backtracking = DFS of state-space tree
8. B&B: FIFO / LIFO / LC search
9. BFS = queue, DFS = stack
10. Back edge ⇔ cycle (directed DFS)
11. Topological order exists ⇔ DAG
12. Kosaraju: DFS, transpose, DFS
13. 4-Queens: 2 solutions; 8-Queens: 92
14. Bellman-Ford $V - 1$ passes + 1 check

Top traps: greedy by ratio is optimal for fractional, not 0/1 knapsack · coin set {1,3,4} breaks greedy · B&B minimisation prunes at LB ≥ best · 1-D knapsack loops w downward · "pre/in/post-order vs level order" is for trees only · Dijkstra fails with a negative edge · Bellman-Ford detects only negative-weight cycles · Floyd-Warshall cannot handle a negative cycle · dense graph → $O(V^2)$ Prim · MST ≠ shortest-path tree · topological order is not unique

All tricks together: "D-C-C: Divide, Conquer, Combine" · "seven, not eight" (Strassen) · "Greedy grabs, DP deliberates" (gold dust vs gold bars) · "Match → diagonal + 1" (LCS) · "Floyd floats through every k" · "Backtracking asks 'is it legal?', B&B asks 'can it still win?'" · "Kahn counts in-degrees, DFS reverses finish times" · "Prim = Proximity, Kruskal = Kheapest edge" · "BFS no weights, Dijkstra positive, Bellman negative, Floyd all pairs".

✓ Practice MCQs (exam pattern, 10 marks)

1. Strassen's matrix multiplication runs in:
(a) $O(n^3)$ (b) $O(n^{2.81})$
(c) $O(n^2 \log n)$ (d) $O(n^2)$

2. The Floyd-Warshall algorithm uses:
(a) greedy (b) divide & conquer
(c) dynamic prog. (d) backtracking

3. Greedy by highest value/weight is always optimal for the ___ knapsack:
(a) 0/1 (b) fractional
(c) bounded (d) multi-constraint

4. Time complexity of Bellman-Ford:
(a) $O(V + E)$ (b) $O(E \log V)$
(c) $O(V^3)$ (d) $O(VE)$

5. Which detects a negative-weight cycle reachable from the source?
(a) Dijkstra (b) Prim
(c) Bellman-Ford (d) Kruskal

6. A topological ordering exists if and only if the graph is:
(a) a DAG (b) complete
(c) bipartite (d) cyclic

7. $T(n) = 2T(n/2) + n$ solves to:
(a) $\Theta(n)$ (b) $\Theta(n \log n)$
(c) $\Theta(n^2)$ (d) $\Theta(\log n)$

8. Edges A→B 4, A→C 1, C→B 2, B→D 1, C→D 5. Shortest distance A to D:
(a) 6 (b) 5
(c) 4 (d) 3

9. An MST of a connected graph with 10 vertices has how many edges?
(a) 10 (b) 45
(c) 11 (d) 9

10. In branch and bound for a minimisation problem, a node is pruned when:
(a) LB ≥ best (b) LB < best
(c) it is a leaf (d) it is the root

Answer key → 1-b · 2-c · 3-b · 4-d · 5-c · 6-a · 7-b · 8-c · 9-d · 10-a

Written question: Four matrices $A_1 \ldots A_4$ have dimensions p = (5, 4, 6, 2, 7). Write the dynamic-programming recurrence for matrix-chain multiplication, fill the cost table, give the optimal parenthesisation and state the time complexity. 10

1. **Recurrence** — $m[i][i] = 0$; $m[i][j] = \min_{i \le k < j} \{ m[i][k] + m[k+1][j] + p_{i-1} p_k p_j \}$. 2
2. **Chains of 2** — $m[1][2] = 5 \cdot 4 \cdot 6 = 120$; $m[2][3] = 4 \cdot 6 \cdot 2 = 48$; $m[3][4] = 6 \cdot 2 \cdot 7 = 84$. 2

3. **Chains of 3** — $m[1][3] = \min(0 + 48 + 5 \cdot 4 \cdot 2 = 88,\ 120 + 0 + 5 \cdot 6 \cdot 2 = 180) = 88$ ($k = 1$); $m[2][4] = \min(0 + 84 + 4 \cdot 6 \cdot 7 = 252,\ 48 + 0 + 4 \cdot 2 \cdot 7 = 104) = 104$ ($k = 3$). 2
4. **Full chain** — $m[1][4] = \min(k{=}1{:}\ 0 + 104 + 140 = 244;\ k{=}2{:}\ 120 + 84 + 210 = 414;\ k{=}3{:}\ 88 + 0 + 70 = 158) =$ **158**, split $k = 3$. 2
5. **Answer** — $((A_1(A_2A_3))A_4)$ with 158 multiplications; time $O(n^3)$, space $O(n^2)$. 2

> **Before the exam ✓:** ☐ recurrences of merge sort, binary search, Strassen · ☐ fractional vs 0/1 knapsack worked once · ☐ LCS and knapsack tables by hand · ☐ 4-Queens backtrack trace · ☐ B&B prune rule both ways · ☐ BFS/DFS order and edge classes · ☐ Kahn's algorithm on a small DAG · ☐ Kruskal and Prim on one graph · ☐ Dijkstra table · ☐ the shortest-path comparison table